```
export async function POST(req) {
    const { messages } = await req.json();
    const transport = new StdioClientTransport({
        command: 'node',
        args: ['src/stdio/server.js'],
    });

    const mcpClient =
await experimental_createMCPClient({ transport });
    const tools = await mcpClient.tools();

    const result = streamText({
        model: gemini('gemini-2.5-flash'),
        messages: convertToModelMessages(messages),
        tools,
        system: 'You are a helpful assistant that can
call tools when needed.',
        onFinish: async () => {
            await mcpClient.close();
        },
        onError: async () => {
            await mcpClient.close();
        },
    });
    return result.toUIMessageStreamResponse();
}
```

Configures stdio client transport to connect to the local MCP server script

Initializes the MCP client using the transport

Retrieves the available tools exposed by the MCP server

Streams the AI-generated response to the frontend in UIMessage format

`StdioClientTransport` is used to facilitate communication with the MCP server over standard input/output (`stdio`) streams. It allows the application to send requests to the server and receive responses in a reliable, streamable manner without needing a separate network connection. The responses from the server are structured in the Gemini protocol format, which represents messages as a sequence of structured events (such as text, tool outputs, or errors). This format enables the frontend to render the chat dynamically, supporting incremental updates and tool integrations while maintaining compatibility with the user interface. The MCP client is closed after the response finishes or if an error occurs, ensuring proper cleanup and modular tool execution.

The following is the relevant snippet for page.js that uses the `useChat` hook:

```
const { messages, sendMessage, status } = useChat({
  transport: new DefaultChatTransport({
    api: '/api/',
  }),
});
```

This hook connects the React frontend to the API route that streams responses from the Gemini model and MCP tools. `messages` contains the chat history, `sendMessage` sends user input to the server, and `status` indicates the current state of the chat (ready, loading, etc.). The transport ensures all messages go through the `/api/` endpoint, allowing the AI to invoke tools seamlessly.